GUDSKY RESEARCH FOUNDATION

Virtual Research Center

Section 8 Non-Profit · ISO Certified · MSME · AICTE · Startup India · NGO Darpan

VIDEO UNDERSTANDING AI

Project Documentation

Technical Reference & Implementation Guide

Document Information	
Project Name	Video Understanding AI
Model	Qwen2.5-VL-7B-Instruct
Hardware	2× NVIDIA L4 GPU (24 GB VRAM each)
Version	1.0.0
Department	Virtual Research Center
Organization	Gudsky Research Foundation
Classification	Research Documentation

Table of Contents

1. Introduction **3**
1.1 About Gudsky Research Foundation 3
1.2 Project Overview 3
1.3 Objectives 3

2. Theoretical Background **4**
2.1 What is Artificial Intelligence? 4
2.2 What is a Large Language Model (LLM)? 4
2.3 What is a Vision-Language Model (VLM)? 4
2.4 Qwen2.5-VL-7B-Instruct — The Model Used 4
2.5 How Video Understanding Works 5
2.6 What is BFloat16 Precision? 5
2.7 What is device_map="auto"? 5

3. System Architecture **6**
3.1 High-Level Architecture 6
3.2 Project Folder Structure 6
3.3 Component Descriptions 6

4. Hardware Specification **8**
4.1 NVIDIA L4 GPU 8
4.2 GPU Memory Layout 8

5. Technology Stack **9**

6. API Reference **10**
6.1 REST Endpoints 10
6.2 WebSocket Protocol 10

7. Authentication System **11**
7.1 Overview 11
7.2 Password Security 11
7.3 Session Tokens 11
7.4 WebSocket Authentication Flow 11

8. Data Files **12**
8.1 users.csv 12
8.2 time_vs_gpu_util.csv 12

9. Deployment Guide **13**
9.1 Prerequisites 13
9.2 Option A — Bare Metal Setup 13
9.3 Option B — Docker 13
9.4 Environment Variables (.env) 13

10. Security Considerations **14**
10.1 Input Validation 14
10.2 Authentication Controls 14
10.3 Resource Limits 14
10.4 Recommended Production Hardening 14

11. Glossary **15**

12. References **18**

1. Introduction

1.1 About Gudsky Research Foundation

Gudsky Research Foundation is a Section 8 Non-Profit Organization dedicated to advancing research and education in the fields of Artificial Intelligence, Data Science, and Emerging Technologies. The Foundation is **ISO Certified**, **MSME Certified**, **AICTE Approved**, and registered under **Startup India** and **NGO Darpan**.

This project was developed at the **Virtual Research Center** — the Foundation's dedicated environment for applied AI research and prototype development.

1.2 Project Overview

The Video Understanding AI system is a real-time, web-based application that allows users to upload a video file and ask natural-language questions about its content. The system uses a state-of-the-art multimodal vision-language model to analyze video frames and generate detailed, contextually accurate answers.

> **What does this system do?**
> A user uploads a video (up to 100 MB). The AI extracts frames from the video, analyzes the visual content, and answers any question the user asks — such as "What is happening in this video?", "How many people are visible?", or "Describe the environment shown."

1.3 Objectives

- Provide a production-ready Video Q&A service deployable on GPU-accelerated infrastructure
- Support up to 10 simultaneous users over WebSocket connections
- Implement secure user authentication with account management
- Log GPU and system resource utilisation over time for performance monitoring
- Deliver an intuitive, browser-based interface requiring no client-side installation

2. Theoretical Background

2.1 What is Artificial Intelligence?

Artificial Intelligence (AI) is a branch of computer science that aims to build systems capable of performing tasks that typically require human intelligence — such as understanding language, recognizing images, and making decisions.

2.2 What is a Large Language Model (LLM)?

A Large Language Model (LLM) is a type of AI model trained on vast amounts of text data. It learns statistical patterns in language, enabling it to generate coherent and contextually relevant text in response to prompts.

Key concepts:

- Tokens: The basic units of text an LLM processes. A token is roughly 3–4 characters.
- Parameters: Internal numerical weights learned during training. More parameters generally means greater capability.
- Inference: The process of using a trained model to generate output from new input.
- Context window: The maximum amount of text (in tokens) the model can consider at once.

2.3 What is a Vision-Language Model (VLM)?

A Vision-Language Model (VLM) extends a language model to understand both images and text simultaneously. It connects a visual encoder — which converts images into numerical representations — to a language model that can reason about those representations and produce text answers.

> **Analogy for beginners**
> Think of a VLM as a person who can both look at a photograph and hold a conversation about it. The 'eyes' are the visual encoder; the 'brain' is the language model. Together, they can answer questions like: What is in this image? What is happening? Who is present?

2.4 Qwen2.5-VL-7B-Instruct — The Model Used

This project uses **Qwen2.5-VL-7B-Instruct**, developed by Alibaba Cloud. It is a 7-billion-parameter open-source vision-language model specifically designed to understand images and video frames, follow instructions, and produce detailed natural-language responses.

Model Property	Value
Model Name	Qwen2.5-VL-7B-Instruct
Parameters	7 Billion
Input Modalities	Text + Images/Video Frames
Output	Natural Language Text

Precision (loaded)	BFloat16 (memory-efficient)
Context Length	Up to 32,768 tokens
License	Apache 2.0 (Open Source)
Benchmark	State-of-the-art on VideoMME, ActivityNet-QA

2.4.1 Why This Model and Not Ollama?

Ollama is a popular tool for running LLMs locally, but it does not currently support native video understanding — it cannot process video frames as input tokens. HuggingFace Transformers, the library used in this project, provides full support for Qwen2.5-VL including video frame ingestion, multi-GPU distribution, and precision control.

2.5 How Video Understanding Works

The model does not process video as a continuous stream. Instead, the pipeline samples frames at regular intervals and passes them as a sequence of images:

1. The video file is loaded using OpenCV, a computer vision library.
2. Up to 32 frames are extracted at evenly-spaced intervals across the video's timeline.
3. Each frame is converted from BGR (OpenCV default) to RGB colour format.
4. The frames, along with the user's question, are passed to the model's processor which converts them into tensors — numerical arrays that the GPU can compute on.
5. The model generates an answer token by token using autoregressive decoding.
6. The generated tokens are decoded back to human-readable text and returned to the user.

2.6 What is BFloat16 Precision?

Neural network weights are typically stored as 32-bit floating-point numbers (FP32). Loading a 7B-parameter model in FP32 would require approximately 28 GB of GPU memory. By using BFloat16 (16-bit brain float), the same model requires only approximately 14–15 GB, fitting comfortably on a single NVIDIA L4 GPU with headroom remaining for inference.

2.7 What is device_map="auto"?

When a model is loaded with **device_map="auto"**, HuggingFace's **accelerate** library automatically distributes model layers across all available GPUs. This project has two NVIDIA L4 GPUs (48 GB combined VRAM). The model weights are loaded primarily on GPU 0, with overflow and KV cache using GPU 1 — maximising memory and throughput without any manual configuration.

3. System Architecture

3.1 High-Level Architecture

The system follows a client-server architecture with three primary layers: the frontend user interface, the backend API server, and the AI inference engine.

Architecture Overview

```
Browser (User) ──WebSocket──▶ FastAPI Server ──▶ Qwen2.5-VL Model
(GPU)                        |                  |                  ▼
▼                users.csv        time_vs_gpu_util.csv
```

3.2 Project Folder Structure

```
video-ai/
├── app.py                    # Application entry point
├── requirements.txt          # Python dependencies
├── Dockerfile                # Container definition
├── setup.sh                  # One-shot install script
├── .env                      # Configuration variables
└── src/
    ├── server.py             # FastAPI: REST endpoints + WebSocket
    ├── websocket_handler.py  # Connection manager (10-user cap)
    ├── backend/
    │   └── auth.py           # Authentication & GPU logging
    ├── frontend/
    │   ├── index.html        # Full web UI (login + chat)
    │   └── logo.jpg          # Organisation logo
    ├── model/
    │   └── video_model.py    # Model loader & inference
    └── data/
        ├── users.csv         # Registered user accounts
        └── time_vs_gpu_util.csv  # Resource utilisation log
```

3.3 Component Descriptions

3.3.1 app.py — Entry Point

Starts the Uvicorn ASGI server. Reads configuration from .env (host, port, model name, upload limits). Uvicorn is an asynchronous server that can handle many simultaneous connections efficiently using Python's asyncio event loop.

3.3.2 src/server.py — API Server

Built with FastAPI, a modern Python web framework. Exposes the following endpoints:

Endpoint	Description
GET /	Redirects browser to the web interface
GET /health	Returns model status, GPU memory, active users
POST /auth/register	Create a new user account

POST /auth/login	Authenticate and receive a session token
POST /auth/logout	Invalidate the session token
POST /upload	Upload a video file (requires auth token)
WS /ws	WebSocket endpoint for real-time Q&A

3.3.3 src/websocket_handler.py — Connection Manager

Manages all active WebSocket connections. Enforces the maximum limit of 10 simultaneous users. When the 11th user attempts to connect, they receive a **WebSocket 1013 (Try Again Later)** close code and an explanatory message. Each session stores the user's uploaded video path and conversation history.

3.3.4 src/backend/auth.py — Authentication & Monitoring

Implements user registration and login using SHA-256 password hashing. Manages session tokens (random 32-byte URL-safe strings) in memory with a 24-hour expiry. Writes all accounts to **users.csv** and logs system resource metrics every 60 seconds and after each inference to **time_vs_gpu_util.csv**.

3.3.5 src/model/video_model.py — AI Inference Engine

Loads Qwen2.5-VL-7B-Instruct from HuggingFace Hub on startup. Implements the **analyze(video_path, question)** method which: extracts frames, builds the model prompt, runs inference on GPU, decodes the output, and clears GPU cache. The model is a module-level singleton — loaded once and reused across all requests.

3.3.6 src/data/ — Persistent Storage

Two CSV files store persistent data:

- users.csv — columns: id, name, email, password_hash, role, created_at, last_login
- time_vs_gpu_util.csv — columns: timestamp, gpu0_util_pct, gpu0_mem_used_gb, gpu0_mem_total_gb, gpu1_util_pct, gpu1_mem_used_gb, gpu1_mem_total_gb, active_users, cpu_pct, ram_used_gb

4. Hardware Specification

4.1 NVIDIA L4 GPU

The system is deployed on a machine equipped with two NVIDIA L4 Tensor Core GPUs. The L4 is designed by NVIDIA for AI inference workloads and supports Ada Lovelace architecture optimisations including FP8 and INT4 quantisation.

GPU Specification	NVIDIA L4 (per card)
VRAM	24 GB GDDR6
FP32 Performance	30.3 TFLOPs
TF32 Performance	62.5 TFLOPs
BF16 / FP16	121 TFLOPs
INT8 Performance	242.5 TOPS
INT4 Performance	485 TOPS
Total VRAM (2× cards)	48 GB
System RAM	96 GB
CPU vCores	24

4.2 GPU Memory Layout

With both GPUs available and **device_map="auto"**, the model distributes as follows:

```
GPU 0  ▒▒▒
```

5. Technology Stack

Component	Technology / Library
AI Model	Qwen2.5-VL-7B-Instruct (HuggingFace Transformers)
Multi-GPU Distribution	HuggingFace Accelerate
Video Frame Extraction	OpenCV (opencv-python-headless)
Web Framework	FastAPI (Python)
ASGI Server	Uvicorn
Real-time Communication	WebSockets (built into FastAPI)
Frontend	Vanilla HTML5 / CSS3 / JavaScript
Authentication	SHA-256 hashing + secure random tokens
Data Storage	CSV files (users.csv, time_vs_gpu_util.csv)
Container	Docker (nvidia/cuda:12.4.1 base image)
Deep Learning Backend	PyTorch 2.3+ (CUDA 12.4)
OS	Ubuntu 22.04 / 24.04

6. API Reference

6.1 REST Endpoints

POST /auth/register

Create a new user account.

Request body (JSON):

```
{
  "name":     "Dr. Jane Smith",
  "email":    "jane@gudsky.org",
  "password": "securepassword"
}
```

Response (200):

```
{ "token": "<session_token>", "message": "Account created successfully." }
```

POST /auth/login

Authenticate with email and password.

```
{ "email": "jane@gudsky.org", "password": "securepassword" }
```

Response (200):

```
{ "token": "<session_token>", "name": "Dr. Jane Smith" }
```

POST /upload?session_id=<id>&token=<token>

Upload a video file (multipart/form-data). Requires both a valid WebSocket session_id and auth token. Maximum file size: 100 MB.

GET /health

Returns server status, model state, GPU memory statistics, and active user count. No authentication required.

6.2 WebSocket Protocol

Connect to **ws://host:8000/ws**. All messages are JSON objects.

Client → Server Messages

Message Type	Description
auth	{ "type": "auth", "token": "<token>" } — Must be sent first
question	{ "type": "question", "question": "What is happening?" }
ping	{ "type": "ping" } — Keepalive heartbeat
clear	{ "type": "clear" } — Clear conversation history

Server → Client Messages

Message Type	Description
connected	Sent on connect — contains session_id
auth_ok	Authentication succeeded — contains user name

auth_fail	Token invalid or expired
thinking	Model has started processing the video
answer	Contains question, answer, and turn number
error	An error occurred — contains message
cleared	Conversation history was cleared
pong	Response to ping

7. Authentication System

7.1 Overview

The system implements a stateless token-based authentication model. When a user logs in or registers, the server generates a cryptographically secure random token (256-bit entropy) and returns it to the client. The client stores this token in the browser's localStorage and includes it with every subsequent request.

7.2 Password Security

Passwords are never stored in plain text. Before saving to **users.csv**, each password is hashed using **SHA-256** — a one-way cryptographic function. This means that even if the CSV file is accessed, original passwords cannot be recovered.

> **What is SHA-256?**
> SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function. It takes any input and produces a fixed-length 64-character hexadecimal string. The same input always produces the same output, but it is computationally infeasible to reverse the process or find two different inputs that produce the same hash.

7.3 Session Tokens

Session tokens are generated using Python's **secrets.token_urlsafe(32)** function, which produces 32 bytes of cryptographically random data encoded as a URL-safe base64 string. Tokens expire after 24 hours. They are stored in a server-side in-memory dictionary and validated on every protected request.

7.4 WebSocket Authentication Flow

7. User completes login via the REST API and receives a token.
8. Browser opens a WebSocket connection to /ws.
9. Server sends { "type": "connected", "session_id": "..." }.
10. Client immediately sends { "type": "auth", "token": "..." }.
11. Server validates token — responds auth_ok or auth_fail.
12. All subsequent messages (questions, uploads) are accepted only after auth_ok.

8. Data Files

8.1 users.csv

Located at **src/data/users.csv**. Each row represents one registered user account.

Column	Description
id	Auto-incrementing integer identifier
name	Full name provided at registration
email	Unique email address (lowercase)
password_hash	SHA-256 hash of the password
role	User role: 'user' (default) or 'admin'
created_at	ISO 8601 UTC timestamp of account creation
last_login	ISO 8601 UTC timestamp of most recent login

8.2 time_vs_gpu_util.csv

Located at **src/data/time_vs_gpu_util.csv**. A new row is appended every 60 seconds by a background thread, and also after each AI inference. This file enables performance monitoring and resource utilisation analysis over time.

Column	Description
timestamp	ISO 8601 UTC timestamp
gpu0_util_pct	GPU 0 compute utilisation (0–100%)
gpu0_mem_used_gb	GPU 0 VRAM used in GB
gpu0_mem_total_gb	GPU 0 total VRAM in GB
gpu1_util_pct	GPU 1 compute utilisation (0–100%)
gpu1_mem_used_gb	GPU 1 VRAM used in GB
gpu1_mem_total_gb	GPU 1 total VRAM in GB
active_users	Number of connected WebSocket sessions
cpu_pct	CPU utilisation across all cores (%)
ram_used_gb	System RAM consumed in GB

9. Deployment Guide

9.1 Prerequisites

- Ubuntu 22.04 or 24.04 (bare metal or VM)
- NVIDIA L4 GPUs with CUDA 12.4 drivers installed
- Python 3.11 (Ubuntu 22.04) or Python 3.12 (Ubuntu 24.04)
- Internet access for downloading model weights (~15 GB, one-time)

9.2 Option A — Bare Metal Setup

```
# 1. Clone or unzip the project
cd video-ai

# 2. Run the setup script (installs deps + downloads model)
bash setup.sh

# 3. Start the server
python app.py

# 4. Open browser
# http://localhost:8000
```

9.3 Option B — Docker

```
# Build the image
docker build -t gudsky-video-ai .

# Run with GPU support
# Mount models_cache as a volume so weights persist across restarts
docker run --gpus all \
  -p 8000:8000 \
  -v $(pwd)/models_cache:/app/models_cache \
  --env-file .env \
  gudsky-video-ai
```

9.4 Environment Variables (.env)

Variable	Default Value & Description
MODEL_NAME	Qwen/Qwen2.5-VL-7B-Instruct — HuggingFace model ID
MODEL_CACHE_DIR	./models_cache — Local path to cache model weights
MAX_NEW_TOKENS	1024 — Maximum tokens in model response
MAX_FRAMES	32 — Frames sampled from each video
HOST	0.0.0.0 — Server bind address
PORT	8000 — Server port
MAX_UPLOAD_SIZE_MB	100 — Maximum video file size in MB
MAX_WS_CONNECTIONS	10 — Maximum simultaneous WebSocket users
TORCH_DTYPE	bfloat16 — Model precision

10. Security Considerations

10.1 Input Validation

- Video uploads are validated for MIME type (must begin with 'video/') and file size (maximum 100 MB).
- Questions are trimmed and validated for non-empty content before inference.
- Session IDs are validated as existing active connections before processing uploads.

10.2 Authentication Controls

- All WebSocket messages beyond the auth handshake are rejected if the session is not authenticated.
- Upload endpoint requires both a valid WebSocket session_id and a valid auth token.
- Session tokens expire after 24 hours and are invalidated on logout.

10.3 Resource Limits

- Hard cap of 10 simultaneous WebSocket connections prevents resource exhaustion.
- Temporary video files are deleted from /tmp when the session disconnects.
- GPU cache is explicitly cleared after each inference using torch.cuda.empty_cache().

10.4 Recommended Production Hardening

- Deploy behind an NGINX reverse proxy with HTTPS/WSS (TLS) enabled.
- Replace CSV storage with a proper database (PostgreSQL, SQLite) for concurrent write safety.
- Add rate limiting per user to prevent inference abuse.
- Rotate the secret key used for token generation and store in environment variables.

11. Glossary

Term	Definition
ASGI	Asynchronous Server Gateway Interface — Python standard for async web servers
BFloat16	A 16-bit floating-point format optimised for AI workloads
CUDA	NVIDIA's parallel computing platform for GPU programming
device_map	HuggingFace parameter for distributing model layers across GPUs
FastAPI	A high-performance Python web framework
GPU	Graphics Processing Unit — specialised for parallel AI computation
HuggingFace	Platform hosting open-source AI models and the Transformers library
Inference	Using a trained model to generate predictions from new input
KV Cache	Key-Value cache storing intermediate attention computations for speed
LLM	Large Language Model — an AI model trained on text data
Multimodal	An AI system that processes multiple types of input (text + images)
OpenCV	Open Source Computer Vision library for image and video processing
Parameters	Learned numerical weights inside a neural network model
REST	Representational State Transfer — standard HTTP API architecture
SHA-256	Cryptographic hash function producing a 256-bit digest
Token (AI)	The smallest unit of text processed by a language model
Token (Auth)	A random string used to authenticate a user session
Transformer	Neural network architecture underlying most modern LLMs
VRAM	Video RAM — GPU memory used to store model weights during inference
VLM	Vision-Language Model — understands both images and text
WebSocket	Protocol for full-duplex, persistent browser-server communication

12. References

- Qwen2.5-VL: https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct
- HuggingFace Transformers: https://huggingface.co/docs/transformers
- HuggingFace Accelerate: https://huggingface.co/docs/accelerate
- FastAPI Documentation: https://fastapi.tiangolo.com
- OpenCV Documentation: https://docs.opencv.org
- NVIDIA L4 GPU Datasheet: https://www.nvidia.com/en-us/data-center/l4
- PyTorch Documentation: https://pytorch.org/docs
- Gudsky Research Foundation: Virtual Research Center

© Gudsky Research Foundation · Virtual Research Center · All Rights Reserved